LaosSA: An Approach Based on Fine-tuning Pre-trained Language Models for Lao Sentiment Analysis

Khamtay Kongmanh[1], Quang-Vinh Pham[2], Quang-Hung Le[1*]

[1] Department of Information Technology, Quy Nhon University, Gia Lai, Vietnam
kongmanh4551050274@st.qnu.edu.vn, lequanghung@qnu.edu.vn

[2] Department of Data Science, TMA Tech Group, Gia Lai, Vietnam
pqvinh@tma.com.vn

Abstract

Effective sentiment analysis for the Lao language is hindered by a scarcity of annotated resources and the inherent noise of real-world user feedback, which often features code-mixing and informal script patterns. This study addresses these challenges by introducing a high-quality, filtered benchmark of 25,139 Lao-language reviews collected from popular digital applications. We evaluate classical baselines, a lightweight textual convolutional neural network model trained from scratch, and several state-of-the-art pre-trained language models, including XLM-RoBERTa, multilingual BERT, and a Lao-specific multilingual BERT, across full fine-tuning, low-rank adaptation, and k-fold cross-validation settings as applicable. Our experimental results demonstrate that XLM-RoBERTa achieves superior performance with 96.64% accuracy and an F_1-macro score of 0.9617. The textual convolutional neural network also provides a strong neural baseline with 94.03% F_1-macro on the fixed split, placing it between the sparse baselines and the strongest pre-trained language model variants. We also find that while Lao-oriented pre-training provides significant benefits, low-rank adaptation offers a highly efficient performance-cost trade-off, maintaining competitive accuracy with minimal trainable parameters. Our research highlights the robustness of multilingual transfer for low-resource sentiment classification and provides a publicly available dataset and code at https://github.com/KT246/LaosSA.

Index Terms

Lao Sentiment Classification, Pre-trained Language Models, TextCNN, LoRA, Low-resource NLP.

1 Introduction

Sentiment analysis helps digital-service providers extract actionable information from large volumes of user feedback [1]–[3]. For Lao, however, large-scale sentiment classification remains limited by the scarcity of annotated corpora, standardized benchmarks, and mature NLP toolchains [4]–[6]. The problem is further complicated by the Lao script itself, which is continuous and non-linear, and by real app-store reviews that are short, noisy, and often code-mixed [5], [7]–[9]. As a result, rule-based and classical methods often require heavy feature engineering, while generic multilingual PLMs may still underrepresent Lao [4]. These constraints motivate a direct comparison among multilingual PLMs (XLM-RoBERTa [10], mBERT [11]), a Lao-supporting model (mBERT-Lao [4]), and lightweight neural and sparse baselines under both full fine-tuning and parameter-efficient adaptation. To address this gap, we introduce *LaosSA*, a benchmark for Lao sentiment classification built from a filtered dataset of 25,139 mobile-application reviews. Our main contributions are:

- We present a filtered Lao sentiment dataset curated from mobile-application reviews and annotated to reflect real colloquial and code-mixed usage.
- We establish a benchmark covering multilingual transfer, Lao-aware pre-training, LoRA [12], sparse baselines, and TextCNN.

- We provide a joint analysis of predictive performance, error patterns, and efficiency trade-offs for practical Lao NLP deployment.

The rest of this paper is structured as follows. Section 2 situates the study within the broader low-resource sentiment analysis and PLM landscape as it relates to broader trends. The construction of the LaosSA dataset and the steps of data collection, annotation and cleaning pipeline is described in Section 3. Section 4 formalizes the classification task and introduces the modelling framework and their refinement steps of fine-tuning. Section 5 presents our configurations of experiments, results and critical discussion. Section 6 ends the paper and presents potential future research directions.

2 Related Work

Lao sentiment analysis has developed along the broader trajectory of multilingual PLMs and low-resource adaptation. mBERT [11] introduced multilingual pre-training, but its shared vocabulary can under-serve Lao and fragment continuous-script inputs. XLM-RoBERTa [10] improved cross-lingual representations, yet it remains a generic multilingual model that does not directly address Lao-specific tokenization or noisy informal reviews.

A related regional example is Vietnamese e-commerce sentiment classification, where recent work reports strong fine-tuning results while also highlighting domain adaptation, informal language, and code-mixing challenges that are relevant to our setting [3]. More broadly, Lao NLP remains constrained by limited corpora, benchmarks, and preprocessing tools [4], [5]. Because Lao is written as a continuous script, preprocessing errors can easily propagate into downstream sentiment models [5].

The introduction of LaoPLM in 2022 marked the first dedicated Lao-oriented pre-trained model [4]. It improved formal Lao text classification, but its evaluation did not fully cover short, noisy mobile-app reviews. More recently, Nguyen et al. [5] expanded Lao resources through semi-supervised word segmentation, again emphasizing data construction rather than sentiment classification in informal review settings.

PLMs now dominate text classification because they provide transferable contextual representations [13], [14]. For Lao, the key question is whether broader multilingual transfer or stronger language fit is more useful in practice [4], [13], [15]. Recent low-resource sentiment studies also show that performance depends not only on architecture, but also on adaptation strategy and dataset characteristics [7]–[9]. LaosSA addresses these gaps by combining scale, noisy review data, and adaptation-focused benchmarking in one setting.

3 Dataset Construction and Analysis

This section details the construction and systematic refinement of the Lao sentiment benchmark, covering data collection from digital-service platforms, the manual annotation process, and the multi-stage filtering pipeline designed to ensure dataset quality.

3.1. Data Collection and Annotation

The reviews used in this study were collected from Google Play applications that are widely used in Laos. The corpus includes comments from banking apps, social media apps, and other digital-service applications rather than from a single e-commerce marketplace. This choice was practical: these applications produced enough Lao user feedback to build a usable sentiment dataset, and they reflect the short mobile reviews that a deployed Lao sentiment system would need to handle.

Each record contains two fields, namely the review text and a binary sentiment label. The final benchmark also retains short code-mixed reviews in which Lao words appear together with English interface terms such as *app*, *bug*, or *OK*, provided that the overall sentiment remains interpretable. This choice keeps the released benchmark closer to real mobile feedback in Laos and directly corresponds to the code-mixing setting referenced in the released resources.

The raw reviews were manually annotated with binary sentiment labels. Each review was assigned a single label based on its overall polarity: label 1 for positive sentiment and label 0 for negative sentiment. The annotation process focused on the dominant sentiment expressed in each review, including explicit praise, complaints, dissatisfaction, or statements about overall user experience.

To improve label consistency, the annotation followed a simple set of practical guidelines. Reviews expressing clearly favorable opinions toward an application, service quality, or user experience were marked as positive, whereas reviews expressing frustration, malfunction, dissatisfaction, or criticism were marked as negative. During dataset refinement, noisy, invalid, or suspicious samples were revisited through the cleaning pipeline described below so that only reliable sentiment instances were retained in the final corpus.

3.2. Data Preprocessing and Cleaning Pipeline

Before final training, the raw reviews were cleaned and filtered in several stages. After manual labeling, we removed noisy content such as repeated characters, emojis, invalid symbols, and strings that did not form meaningful Lao text. Particular attention was paid to Lao-specific character errors, especially cases in which vowels appeared without consonants or in which character combinations were visually present but linguistically invalid.

To assess label quality more efficiently, we trained a lightweight screening model and compared its predictions with the manual annotations. Samples that repeatedly agreed with their labels were retained, whereas mismatched cases were rechecked and many were removed. This step served as a practical filter for suspicious or low-quality reviews before the final benchmark experiments.

We also used a separate 5-fold cross-validation auditing step for difficult cases during dataset cleaning. This step was part of data filtering only and is distinct from the 3-fold cross-validation used later for model evaluation. Reviews that were repeatedly misclassified across folds were treated as unstable, and many of them contained invalid Lao character patterns, lexical inconsistencies, or meaningless text fragments.

After this filtering process, the remaining reviews formed the final dataset used for training and validation.

3.3. Dataset Statistics

Table I summarizes the final train and validation splits after cleaning and filtering. The two partitions remain closely matched in class balance and contain no missing text or labels. The reviews are short on average, with only a small number of much longer comments, which is consistent with the brief and noisy style of app-store feedback.

TABLE I
TRAIN AND VALIDATION STATISTICS AFTER FILTERING AND PREPROCESSING.

Statistic	**Train**	**Val.**	**Statistic**	**Train**	**Val.**
Rows	20,111	5,028	Columns	2	2
Empty text	0	0	Empty label	0	0
Unique text	20,095	5,028	Duplicated text	16	0
Negative reviews (Label = 0)	6,312 (31.39%)	1,578 (31.38%)	Positive reviews (Label = 1)	13,799 (68.61%)	3,450 (68.62%)
Min. text length (char.)	2	3	Min. word count	1	1
Avg. text length (char.)	29.41	30.29	Avg. word count	1.52	1.52
Max. text length (char.)	618	506	Max. word count	66	77

4 Methodology

This section presents the formal task definition for Lao sentiment classification, introduces the selected PLMs, and describes the fine-tuning strategies and training objectives employed in our modeling framework.

4.1. Problem Definition

We formulate Lao sentiment classification as a supervised binary text classification task over cleaned Lao reviews. Let the benchmark dataset be defined as

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}, \tag{1}$$

where each review $x_i = (w_1, w_2, \ldots, w_n)$ is represented by the token sequence produced by a given tokenizer, and each label satisfies $y_i \in \{0, 1\}$, with 0 denoting negative sentiment and 1 denoting positive sentiment. Each review receives a single sentence-level label corresponding to its dominant polarity, even when the text remains short, informal, or noisy.

The task is to learn a classifier f_ψ that maps a Lao review to one of the two sentiment classes. For each input x, a model estimates a posterior distribution over the label space and predicts the dominant polarity according to

$$\hat{y} = f_\psi(x) = \arg \max_{c \in \{0,1\}} p_\psi(y = c \mid x), \tag{2}$$

where ψ denotes the parameters of either a classical classifier or an adapted PLM-based classifier, and c indexes the candidate sentiment class. This formulation keeps the task definition model-agnostic so that sparse baselines and PLM-based systems can be compared under the same prediction rule.

The benchmark is defined under several practical assumptions. Each review is treated as an independent sample without conversational context or user metadata. The experiments focus on document- or sentence-level polarity classification rather than aspect-level sentiment extraction, and all systems operate directly on the cleaned review text rather than on hand-crafted linguistic features. These assumptions keep the comparison centered on representation quality and adaptation strategy rather than on feature design or external metadata.

The evaluation protocol follows the final filtered split described in Section 3.3. For the main benchmark, models are trained on 20,111 training samples and evaluated on a fixed validation set of 5,028 samples. In addition, a separate 3-fold cross-validation setting is used as a robustness check on the training partition. Performance is reported using Accuracy, Precision-macro, Recall-macro, and F_1-macro, with F_1-macro treated as the primary comparison metric because of the moderate class imbalance between positive and negative reviews.

4.2. Pre-trained Language Models

We evaluate three PLMs: XLM-RoBERTa [10], mBERT [11], and mBERT-Lao [4]. XLM-RoBERTa and mBERT serve as multilingual references for cross-lingual transfer [13]–[15], while mBERT-Lao provides a Lao-supporting alternative [4]. Table II summarizes the main properties of the three models under the benchmark-facing names used throughout this paper. All PLM runs share the same preprocessing and evaluation protocol, and the LoRA setting attaches adapters to the query, key, and value projections.

In addition to the PLMs, the benchmark includes four non-PLM references: Logistic Regression, Support Vector Machine (SVM), Decision Tree [16], and TextCNN [17]. The first three use Term Frequency-Inverse Document Frequency (TF-IDF) character n-gram features, while TextCNN is evaluated under from-scratch training as a compact convolutional classifier with 128-dimensional embeddings, 128 filters per kernel size $\{3, 4, 5\}$, and dropout 0.3. This gives the benchmark a lightweight neural baseline that does not rely on pretrained language-model weights and allows for a direct comparison between transfer-learning and static initialization paradigms.

4.3. Fine-Tuning and From-Scratch Training Strategies

Each PLM is trained with either Full Fine-tuning or LoRA, while the neural baseline is trained from-scratch. Full Fine-tuning updates the backbone and classification head end to end. LoRA instead freezes the pretrained weights and learns low-rank updates

TABLE II
PRE-TRAINING AND ARCHITECTURAL COMPARISON OF THE PLMS USED IN THE BENCHMARK.

Aspect	**XLM-RoBERTa**	**mBERT**	**mBERT-Lao**
Language scope	Multilingual (100 languages)	Multilingual (104 languages)	Lao-supporting
Pre-training data	CommonCrawl multilingual corpus (∼2.5 TB)	Multilingual Wikipedia corpus	Lao-oriented RoBERTa-style pretraining
Tokenizer	SentencePiece, ∼250k subwords	WordPiece, ∼119k vocabulary	RoBERTa tokenizer, ∼50k vocabulary
Architecture (base)	12 layers, 768 hidden, 12 heads	12 layers, 768 hidden, 12 heads	12 layers, 768 hidden, 12 heads
Role in this benchmark	Strong multilingual reference	Strong multilingual reference	Lao-supporting comparison model

in selected attention projections [12], [18], [19]. From-scratch training involves random weight initialization and localized optimization without knowledge transfer. We also report 3-fold cross-validation as a robustness check; in that setting, a separate model is trained on each fold and the final score is computed from the concatenated out-of-fold predictions.

Let θ denote the backbone parameters and ϕ the classification head. Full Fine-tuning solves

$$(\theta^*, \phi^*) = \arg\min_{\theta,\phi} \mathcal{L}_{\text{train}}(\theta, \phi). \tag{3}$$

For LoRA, the pretrained matrix is decomposed as $W = W_0 + BA$, where W_0 is frozen and $A \in \mathbb{R}^{r\times d}$ and $B \in \mathbb{R}^{d\times r}$ are trainable. The optimization becomes

$$(A^*, B^*, \phi^*) = \arg\min_{A,B,\phi} \mathcal{L}_{\text{train}}(W_0 + BA, \phi). \tag{4}$$

For cross-validation, out-of-fold predictions from all K folds are concatenated and scored jointly. The reported score is therefore

$$\text{Score}_{\text{CV}} = \text{Score}\left(\bigcup_{k=1}^{K} \hat{D}_{\text{val}}^{(k)}\right), \quad K = 3, \tag{5}$$

where $\hat{D}_{\text{val}}^{(k)}$ denotes the predictions produced by the best checkpoint of fold k on its held-out partition.

Fig.1 summarizes the benchmark in three stages. The workflow begins with Google Play review collection, binary annotation, normalization, cleaning, and the post-processing audit used to construct the final split. It then trains XLM-RoBERTa, mBERT, mBERT-Lao, TextCNN, and TF-IDF baselines under the strategy variants applicable to each family, including Full Fine-tuning, LoRA, from-scratch training, and 3-fold cross-validation. Finally, the pipeline exports checkpoints, validation predictions, standard metrics, timing summaries, and false-positive/false-negative artifacts for the later analysis sections.

4.4. Training Objective

PLM runs minimize weighted cross-entropy loss. Class weights compensate for the moderate label imbalance in the final dataset. For a training set of N samples and two sentiment classes, the objective is

$$\mathcal{L} = -\sum_{i=1}^{N}\sum_{c=1}^{2} w_c y_{i,c} \log \hat{y}_{i,c}, \tag{6}$$

where w_c is the weight for class c, $y_{i,c}$ is the one-hot target, and $\hat{y}_{i,c}$ is the predicted probability for class c.

We train the PLMs with standard pretrained-model tooling and fine-tuning practice [13], [14], using a learning rate of $1e^{-5}$, weight decay 0.01, warmup ratio 0.1, and maximum sequence length 128. For Full Fine-tuning and LoRA, the best checkpoint is selected by validation loss. TextCNN uses the same epoch budget, batch sizes, and sequence length, but it initializes all parameters from scratch. The classical baselines use the same split and train TF-IDF character n-gram features from 3 to 5 with `min_df`=2 and at most 50,000 features before classifier fitting.

Algorithm 1 summarizes the unified benchmark pipeline used throughout the experiments. It reflects the fixed-split baseline runs, the PLM-based fixed-split runs, and the separate 3-fold cross-validation setting reported later in Section 5.

5 Experiments

This section outlines the experimental configuration, baseline selections, and evaluation metrics. We then present a comparative analysis of our findings regarding predictive performance, error patterns, and computational efficiency across the evaluated models.

5.1. Experimental Setup

We begin by specifying the compared systems, the evaluation metrics, and the implementation details required to reproduce the benchmark.

5.1.1. Compared Models and Implementation Details

The benchmark combines classical baselines, a lightweight neural baseline, and PLM-based models. The classical group contains Logistic Regression, SVM, and Decision Tree [16]. The neural baseline is TextCNN [17], evaluated under from-scratch training and 3-Fold Cross-Validation. The PLM group contains XLM-RoBERTa [10], mBERT [11], and mBERT-Lao [4], each evaluated with Full Fine-tuning, LoRA [12], and 3-Fold Cross-Validation. This setup creates a controlled comparison between sparse models, a compact neural model, and pretrained transformers on the same Lao review data.

The classical baselines were trained on the same fixed train-validation split used for the fixed-split PLM runs. They use character-level TF-IDF features with an n-gram range of 3-5, `min_df`=2, and at most 50,000 features. Logistic Regression and SVM were implemented through `SGDClassifier` with `log_loss` and `hinge` objectives, respectively, while Decision Tree used `max_depth`=40 and `min_samples_leaf`=2. TextCNN uses 128-dimensional trainable embeddings, 128 filters for each kernel size in $\{3, 4, 5\}$, and dropout

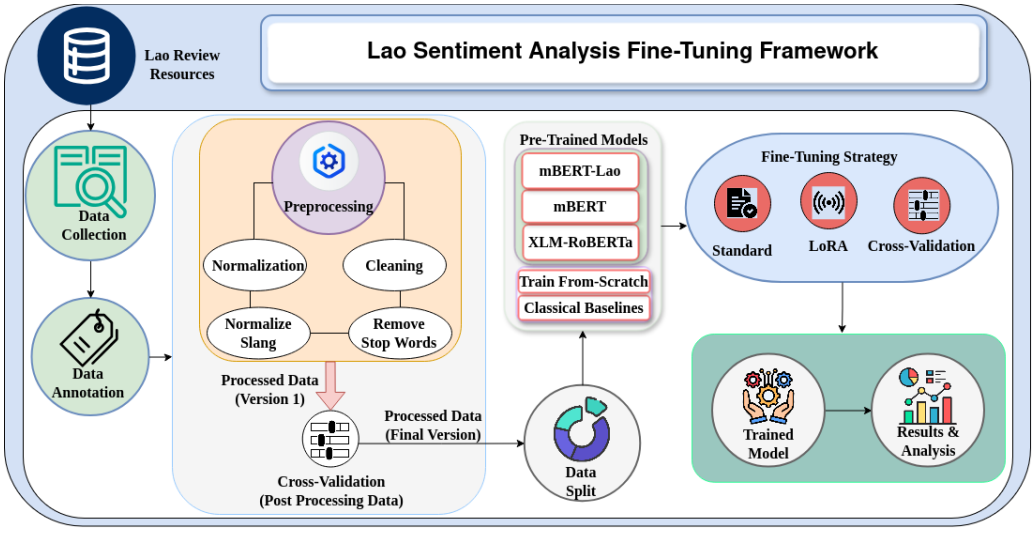

Fig. 1. Overall methodology framework for Lao sentiment classification, including dataset preparation, model selection, and training strategies.

Algorithm 1 Unified training and evaluation pipeline for Lao sentiment classification

Require: fixed-split $D_{\text{train}}, D_{\text{val}}$, PLM set $\mathcal{M}$, baseline set $\mathcal{B}$, neural reference set $\mathcal{N}$, strategies $\mathcal{S} = \{\text{FT}, \text{LoRA}\}$
Ensure: prediction files, metrics, timing statistics, and error-analysis artifacts

for all $b \in \mathcal{B}$ **do** ▷ Classical sparse baselines
  compute TF-IDF features $\Phi(x)$ for $x \in D_{\text{train}}$
  $\psi_b \leftarrow \arg\min_\psi \mathcal{L}_{\text{train}}(\psi, \Phi(D_{\text{train}}))$
  $\hat{y} \leftarrow f_{\psi_b}(\Phi(x))$ for $x \in D_{\text{val}}$, evaluate metrics
end for
for all $n \in \mathcal{N}$ **do** ▷ Neural models (from-scratch)
  randomly initialize ψ_n, minimize $\mathcal{L}$ on D_{train} via Eq. (6)
  $\psi^* = \arg\min_\psi \mathcal{L}(\psi, D_{\text{val}})$
  $\hat{y} = f_{\psi^*}(x)$ for $x \in D_{\text{val}}$, store metrics
end for
for all $m \in \mathcal{M}$ **do** ▷ PLMs (fine-tuning)
  for all $s \in \mathcal{S}$ **do**
    load tokenizer and pretrained checkpoint for m
    if $s = \text{LoRA}$ **then**
      inject LoRA adapters BA via Eq. (4)
    else
      enable end-to-end updates via Eq. (3)
    end if
    minimize $\mathcal{L}$ on D_{train} via Eq. (6)
    $\psi^* = \arg\min_\psi \mathcal{L}(\psi, D_{\text{val}})$
    compute $\hat{y} = f_{\psi^*}(x)$ for $x \in D_{\text{val}}$, store metrics
  end for
  for $k = 1$ to 3 **do**
    partition $D_{\text{train}} \to \{D_{\text{train}}^{(k)}, D_{\text{val}}^{(k)}\}$
    $\psi^{(k)} \leftarrow \text{optimize}(\mathcal{L}, D_{\text{train}}^{(k)}, m)$
    $\hat{D}_{\text{val}}^{(k)} \leftarrow \{f_{\psi^{(k)}}(x) \mid x \in D_{\text{val}}^{(k)}\}$
  end for
  compute Score_{CV} using Eq. (5)
end for
aggregate $\hat{D}_{\text{val}}$ and metrics to produce final artifacts

0.3, giving the benchmark a compact neural reference that does not depend on pretrained weights.

5.1.2. Evaluation Metrics

We report Accuracy, Precision-macro, Recall-macro, and F_1-macro, which are standard evaluation measures for text classification and information retrieval tasks [1], [20]. Because the dataset is moderately imbalanced, F_1-macro is treated as the main comparison metric, while the remaining metrics are used to show overall correctness and class-level balance.

For binary sentiment classification, Accuracy is defined as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{7}$$

For each class $c \in \{0, 1\}$, let

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}. \tag{8}$$

The macro-averaged scores reported in this paper are then

$$\text{Precision}_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} P_c, \quad \text{Recall}_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} R_c, \quad (F_1)_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} \frac{2P_cR_c}{P_c + R_c}, \tag{9}$$

where c indexes a sentiment class and $C = 2$ for the negative and positive labels. Here, TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively, while TP_c, FP_c, and FN_c are the corresponding class-specific counts used to compute P_c and R_c for class c.

The experiments use the final filtered Lao review dataset described in Section 3.3. Logistic Regression, SVM, Decision Tree, and fixed-split TextCNN were trained on the same validation split as the fixed-split PLM runs, while 3-fold cross-validation was applied on the training partition for the PLMs and for TextCNN. The exact hyperparameters are listed in Table III; the main difference across PLM runs is whether the full backbone is updated or only LoRA adapters are trained. Runtime metadata recorded across 14 experiment folders indicate a Tesla T4 GPU with 14.56 GB GPU memory, 12.67 GB RAM, and 1 physical CPU core (2 total cores).

TABLE III
TRAINING CONFIGURATION USED IN THE EXPERIMENTS.

Component	**Configuration**
PLM setup	25 epochs, max length 128, validation loss
Batch sizes	train 16, eval 32
Optimization	lr 1×10^{-5}, weight decay 0.01, warmup 0.1
Cross-validation	Stratified $K = 3$
LoRA setting	$r = 16$, $\alpha = 32$, dropout 0.1
TextCNN setting	embedding 128, filters 128, kernels 3/4/5, dropout 0.3
Baseline features	TF-IDF char n-grams (3–5), max 50,000

5.2. Results and Discussion

We now summarize the main performance pattern before turning to error behavior and efficiency trade-offs.

5.2.1. Overall Performance and Domain Adaptation

Table IV and Fig. 2 show that backbone suitability matters more than simply moving from sparse to neural baselines, or from LoRA to Full Fine-tuning within the same PLM family. XLM-RoBERTa remains the strongest model family, mBERT-Lao clearly outperforms generic mBERT, and TextCNN provides a useful middle point between sparse baselines and the best PLMs. This pattern is consistent with low-resource sentiment studies that emphasize language fit and dataset characteristics, not only the number of updated parameters [7], [8], [21]. The competitiveness of SVM and Logistic Regression also indicates that many Lao app-review decisions still rely on short lexical sentiment cues.

TABLE IV
OVERALL PERFORMANCE GROUPED BY MODEL FAMILY AND TRAINING STRATEGY.

Model Family	**Training Strategy**	**Acc.**	F_1	**Prec.**	**Rec.**
XLM-RoBERTa	Full Fine-tuning	**0.9664**	**0.9617**	**0.9544**	**0.9702**
	LoRA	0.9642	0.9589	0.9539	0.9645
	Cross-Validation	0.9648	0.9597	0.9541	0.9659
mBERT	Full Fine-tuning	0.6969	0.6057	0.6336	0.6012
	LoRA	0.6901	0.6048	0.6253	0.6004
	Cross-Validation	0.6967	0.6137	0.6350	0.6084
mBERT-Lao	Full Fine-tuning	0.9407	0.9328	0.9241	0.9437
	LoRA	0.9326	0.9235	0.9154	0.9335
	Cross-Validation	0.9375	0.9285	0.9228	0.9350
TextCNN	From Scratch	0.9477	0.9403	0.9338	0.9478
	Cross-Validation	0.9339	0.9242	0.9192	0.9298
Classical baselines	Logistic Regression	0.9451	0.9370	0.9322	0.9425
	SVM	0.9505	0.9436	0.9363	0.9522
	Decision Tree	0.8878	0.8721	0.8662	0.8792

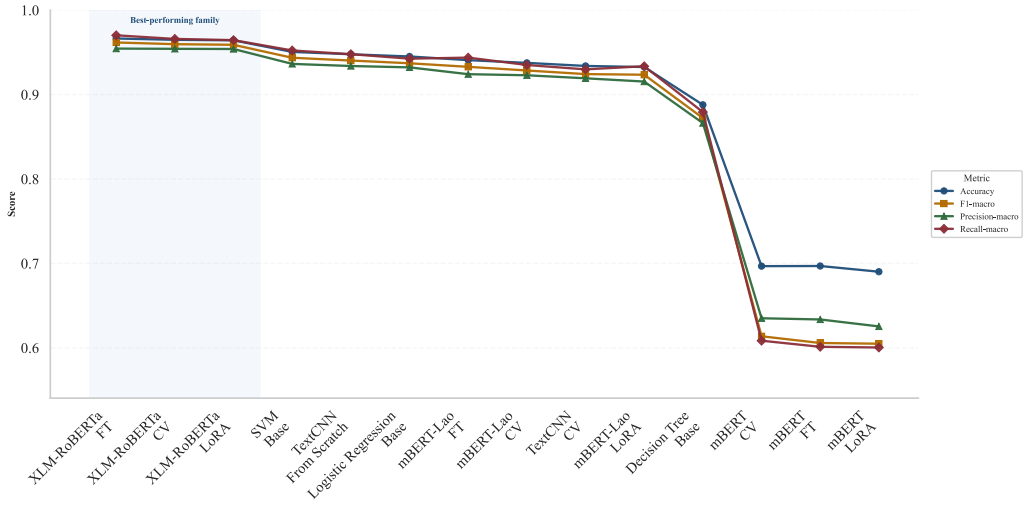

Fig. 2. Metric comparison across all benchmarked settings, sorted by F_1-macro.

The gap between mBERT-Lao and generic mBERT suggests that Lao support is a substantive source of performance gain rather than a minor implementation detail. This aligns with prior LaoPLM-style findings that multilingual pretraining can under-serve Lao when tokenization and corpus coverage are mismatched [4], [21]. TextCNN further shows that a compact from-scratch model can learn review-local patterns effectively, although its lower cross-validation scores suggest greater sensitivity to data variation than the strongest PLMs. Overall, domain adaptation in this benchmark depends on both language-aware representations and sensitivity to concise, review-specific expressions.

5.2.2. Error Analysis

Fig.3 helps explain why the model ranking separates so clearly. The strongest systems maintain a more balanced error profile, whereas weaker multilingual transfer is associated with a clear positive-class bias. This matters because the difficult cases are not random; they are concentrated in short complaint-like comments, code-mixed fragments, and mixed-polarity reviews that combine a problem report with a request, update, or partially positive tone. Related low-resource sentiment studies likewise report that short, informal, and mixed-language comments are a recurring source of ambiguity [7], [8], but the present results suggest that these cases are especially consequential for Lao app reviews, where limited context and informal phrasing amplify ambiguity.

Table V illustrates this pattern qualitatively. False positives are usually short complaints whose negative intent is expressed indirectly or through code-mixed wording, whereas false negatives more often contain softened criticism, requests, or mixed cues. The dominant failure mode is therefore semantic ambiguity in short feedback, rather than annotation instability alone.

Fig.4 provides a phrase-level view of the same error set using the 30 most frequent short false-positive and false-negative samples.

TABLE V
REPRESENTATIVE EXAMPLES OF FIXED-SPLIT ERROR PATTERNS

False Positive		False Negative	
Example	**Label**	**Example**	**Label**
ບໍ ok *not OK*	0	ເປັນຍັງຄື ທັ້ງໝົດທີ່ຈຫຼີ້ນບໍ່ໄດ້ *why does nothing work?*	1
ປ່ຽນໂທລະສັບໃຫມ່ *changed to a new phone*	0	ຄັກຢູ່ ແຕ່ໃຫ້ອັບເດດດຸໂພດ *it is good, but please update it*	1
ເບີລະຫັດ whatsapp *WhatsApp verification code*	0	ແຕ່ໃຫ້ປັບປຸງໃຫ້ໄວກ່ວາເກົ່າ *please make it faster*	1
bug ຫລາຍ *too many bugs*	0	ໂຄ້ດດີ ຖ້າບໍ່ອັບເດດແອປ ແຮງຊິດີ *the code is good, but the update hurts the app*	1
ແອັບຊ່ຳສຸດ *the app is very slow*	0	ຂອບເຂດສົ່ງ ກ້ວາງໄກລ ສົ່ງໄວ ບໍ່ມີບັນຫາ *wide service area, fast delivery, no problems*	1

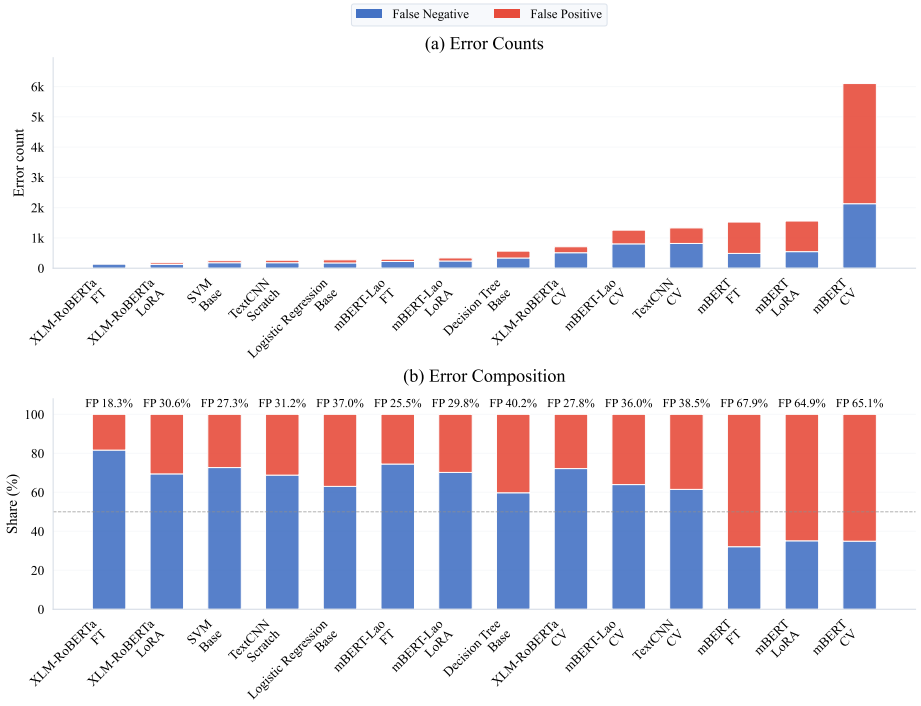

Fig. 3. Aggregated false-negative/false-positive counts and error composition across experiments.

(a) False Positives (b) False Negatives

Fig. 4. Word clouds for the 30 most frequent short failure cases. Larger phrases indicate higher frequency.

5.2.3. Practical Implications

From a deployment perspective, XLM-RoBERTa is the strongest choice when predictive quality is the main priority. TextCNN offers a practical middle ground, reaching 0.9403 F_1-macro on the fixed split in 11.28 minutes. Table VI shows that LoRA cuts the trainable footprint by roughly two orders of magnitude and substantially reduces PLM training time, while the F_1 drop remains small for XLM-RoBERTa. SVM remains a credible low-cost baseline when infrastructure constraints dominate.

TABLE VI
FT VS. LORA EFFICIENCY COMPARISON.

Model Family	Strat.	Params	Ratio	Epoch (s)	Total (m)	F_1
XLM-RoBERTa	FT	278.0M	1.0000	267.3	111.4	**0.9617**
	LoRA	1.48M	0.0053	117.8	49.1	0.9589
mBERT	FT	177.9M	1.0000	205.8	85.8	0.6057
	LoRA	**0.89M**	**0.0050**	116.9	48.7	0.6048
mBERT-Lao	FT	124.6M	1.0000	177.5	74.0	0.9328
	LoRA	1.48M	0.0117	**113.7**	**47.4**	0.9235

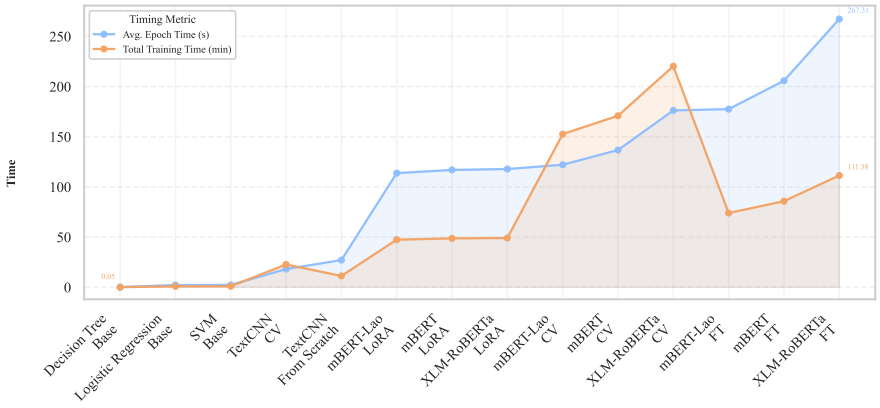

Fig. 5. Timing comparison across benchmark settings.

Fig. 5 extends this pattern to the full benchmark: classical baselines are cheapest, TextCNN is the fastest neural model, and LoRA remains faster than Full Fine-tuning for every PLM family.

6 Conclusion

This study establishes a benchmark for Lao sentiment classification using a filtered dataset of real-world mobile-application reviews. XLM-RoBERTa provides the strongest overall performance, while language fit and adaptation strategy remain decisive for Lao NLP. Lao-aware or stronger cross-lingual transfer outperforms generic multilingual baselines, LoRA preserves much of the PLM quality at lower training cost, and TextCNN remains a practical lightweight reference for noisy, code-mixed reviews. Future work should expand dataset diversity, improve label consistency, and explore finer-grained settings such as aspect-level sentiment analysis.

References

[1] K. L. Tan, C. P. Lee, and K. M. Lim, "A survey of sentiment analysis: Approaches, datasets, and future research," *Applied Sciences*, vol. 13, no. 7, p. 4550, 2023.

[2] K. Alahmadi, S. Alharbi, J. Chen, and X. Wang, "Generalizing sentiment analysis: a review of progress, challenges, and emerging directions," *Social Network Analysis and Mining*, vol. 15, no. 1, p. 45, 2025.

[3] Q.-V. Pham, N.-H. Dang, P.-L. Tran, and Q.-H. Le, "Fine-tuning pre-trained language models for vietnamese sentiment classification in e-commerce," in *Computational Intelligence in Engineering Science*, N. T. Nguyen, A.-C. Le, T. D. Tran, T.-P. Hong, Y. Manolopoulos, N. A. Le Khac, P. Tran Tin, and V.-N. Huynh, Eds. Cham: Springer Nature Switzerland, 2026, pp. 160–174.

[4] N. Lin, Y. Fu, C. Chen, Z. Yang, and S. Jiang, "Laoplm: Pre-trained language models for lao," in *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 2022, pp. 6506–6512.

[5] H. T. Nguyen, T. Palongve, C. Q. Nguyen, and K. T. Le, "Semi-automatic construction and benchmarking of a word-segmented corpus for lao using llms and transformer models," *Informatica*, vol. 49, no. 27, 2025.

[6] W. Phatthiyaphaibun, "Laonlp: Lao language natural language processing," Jul. 2022. [Online]. Available: https://doi.org/10.5281/zenodo.6833407

[7] M. Subramanian, K. Shanmugavadivel *et al.*, "Kec_tech_titans@ dravidianlangtech 2025: Sentiment analysis for low-resource languages: Insights from tamil and tulu using deep learning and machine learning models," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 278–282.

[8] M. Chowdhury, A. Laskar, T. Ahmad, and A. T. Wasi, "Mysticciol@ dravidianlangtech 2025: A hybrid framework for sentiment analysis in tamil and tulu using fine-tuned sbert embeddings and custom mlp architectures," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 167–172.

[9] T. Durairaj, B. R. Chakravarthi, A. Hegde, H. L. Shashirekha, R. Natarajan, S. Thavareesan, R. Sakuntharaj, C. Rajkumar, P. Shetty, H. S. Kumar *et al.*, "Overview of the shared task on sentiment analysis in tamil and tulu," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 732–738.

[10] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8440–8451.

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[13] X. Han, Z. Zhang, N. Ding, Y. Gu, X. Liu, Y. Huo, J. Qiu, Y. Yao, A. Zhang, L. Zhang *et al.*, "Pre-trained models: Past, present and future," *AI open*, vol. 2, pp. 225–250, 2021.

[14] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.

[15] S. Ruder, N. Constant, J. Botha, A. Siddhant, O. Firat, J. Fu, P. Liu, J. Hu, D. Garrette, G. Neubig *et al.*, "Xtreme-r: Towards more challenging and nuanced multilingual evaluation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 10 215–10 245.

[16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.

[17] Y. Kim, "Convolutional neural networks for sentence classification," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1746–1751.

[18] Y. Mao, Y. Ge, Y. Fan, W. Xu, Y. Mi, Z. Hu, and Y. Gao, "A survey on lora of large language models," *Frontiers of Computer Science*, vol. 19, no. 7, p. 197605, 2025.

[19] S. Nwaiwu, "Parameter-efficient fine-tuning for low-resource text classification: a comparative study of lora, ia3, and reft," *Frontiers in big Data*, vol. 8, p. 1677331, 2025.

[20] J. Opitz, "A closer look at classification evaluation metrics and a critical reflection of common evaluation practice," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 820–836, 2024.

[21] N. Z. Lamin and A. A. Aziz, "Cross-lingual sentiment analysis in low-resource languages: A recent review on tasks, methods and challenges." *International Journal of Advanced Computer Science & Applications*, vol. 16, no. 11, 2025.